
ClimbIt: Herramienta para gestionar y gamificar
el progreso en rocódromos
ClimbIt: Tool to manage and gamify progress on
climbing walls



Trabajo de Fin de Grado
Curso 2025–2026

Autores

Iván Alcalde Cámara
Ignacio Melendo Bruggeman

Director

Iván Martínez Ortiz

Grado en Ingeniería del Software
Facultad de Informática
Universidad Complutense de Madrid

ClimbIt: Herramienta para gestionar y
gamificar el progreso en rocódromos
ClimbIt: Tool to manage and gamify
progress on climbing walls

Trabajo de Fin de Grado en Ingeniería del Software

Autores

**Iván Alcalde Cámara
Ignacio Melendo Bruggeman**

Director

Iván Martínez Ortiz

Convocatoria: Junio de 2026

**Grado en Ingeniería del Software
Facultad de Informática
Universidad Complutense de Madrid**

25 de Mayo de 2026

Dedicatoria

*A la maravillosa cafetería de la Facultad de
Informática, por ser el lugar donde se gestó
esta memoria y por su inestimable
contribución a lo largo de todo el proceso.*

Agradecimientos

A todos los que nos han acompañado por nuestro lento pero bonito paso por la universidad.

Resumen

ClimbIt: Herramienta para gestionar y gamificar el progreso en rocódromos

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

ClimbIt: Tool to manage and gamify progress on climbing walls

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Contexto de la escalada en rocódromo	1
1.1.1. Funcionamiento de la escalada en rocódromo	1
1.1.2. Tipos de rutas en rocódromos	1
1.1.3. Dificultad y progreso	2
1.2. Contexto de la gamificación en deportes	2
1.2.1. Modelo psicológico RAMP	2
1.2.2. El sistema PBL (<i>Points, Badges, Leaderboards</i>)	3
1.2.3. Beneficios de la gamificación en el deporte	3
1.3. Motivación	4
1.4. Objetivos	4
1.4.1. Objetivo general	4
1.4.2. Hitos específicos	4
1.5. Estructura del documento	5
2. Estado del Arte	7
2.1. Análisis de aplicaciones de escalada	7
2.1.1. Aplicaciones integradas con el rocódromo	7
2.1.2. Aplicaciones centradas en la comunidad	8
2.1.3. Aplicaciones con hardware específico	8
2.2. Análisis de aplicaciones de otros deportes	9
2.2.1. Competiciones y ranking por segmentos (Strava)	9
2.2.2. Coleccionismo de logros (Pokémon Go)	9
2.3. Conclusiones del Estado del Arte e identificación de la oportunidad	10
2.3.1. Síntesis de la problemática actual	10
2.3.2. Identificación de la oportunidad	10
3. Metodología de Desarrollo y Planificación	13
3.1. Metodología de trabajo: Scrumban	13
3.2. Proceso de Captura de Requisitos y Definición del Producto	13
3.2.1. Lienzo de Propuesta de Valor (Value Proposition Canvas)	14
3.2.2. Visualización del viaje: User Story Map	14

3.3.	Herramientas de gestión y seguimiento	15
3.4.	Gestión de la Configuración y Control de Versiones	15
3.5.	Uso de Inteligencia Artificial como Apoyo al Desarrollo	16
4.	Análisis y Especificación de Requisitos	17
4.1.	Personas	17
4.1.1.	Persona 1: Klaudia, Escaladora Constante	17
4.1.2.	Persona 2: Victor, Gestor del Rocódromo	18
4.2.	Roles	19
4.2.1.	Escalador	19
4.2.2.	Gestor de Rocódromo	19
4.2.3.	Propietario	19
4.2.4.	Administrador	19
4.3.	Especificación de funcionalidades (Historias de Usuario)	20
4.3.1.	Autenticación y Gestión de Perfil	20
4.3.2.	Social y Comunidad	20
4.3.3.	Rocódromos	21
4.3.4.	Zonas	21
4.3.5.	Rutas	21
4.4.	Requisitos no funcionales	22
4.5.	Alcance de las versiones	22
4.5.1.	Requisitos del prototipo (v.0.1.0)	23
4.5.2.	Versiones intermedias (v.0.2.0 - v.0.2.2)	23
4.5.3.	Requisitos del MVP (v.1.0.0)	23
4.5.4.	Mantenimiento y mejoras post-lanzamiento (v.1.0.1)	24
5.	Diseño del Sistema y Decisiones Técnicas	25
5.1.	Decisiones de Diseño y Stack Tecnológico	25
5.2.	Diseño de la Arquitectura	27
5.2.1.	Backend	27
5.2.2.	Frontend	28
5.3.	Diseño de la Base de Datos	28
5.3.1.	Entidades Principales del Sistema	29
5.3.2.	Relaciones Principales del Modelo	29
5.3.3.	Normalización y Diseño	30
5.3.4.	Decisiones de Diseño Específicas	30
5.3.5.	Migraciones y Versionado	31
5.4.	Diseño de la UI y UX	31
6.	Implementación y Desarrollo	33
6.1.	Desarrollo del Backend	33
6.2.	Desarrollo del Frontend	33
6.3.	Desafíos técnicos encontrados y soluciones	33

7. Integración Continua, Despliegue y DevOps	35
7.1. Diseño la Integración Continua	35
7.1.1. Calidad de Código y Validación Pre-commits	35
7.1.2. Workflow de Integración (ClimbIt-CI)	36
7.2. Entornos de desarrollo, preproducción y producción	36
7.3. Despliegue de la infraestructura	37
7.3.1. Orquestación mediante Contenedores	37
7.3.2. Hardware y Ejecución (Self-hosted Runner)	37
7.3.3. Exposición Segura y Cifrado: Cloudflare Zero Trust	37
7.4. Empaquetado y Subida a Google Play	38
7.4.1. Tecnología Trusted Web Activity (TWA) y Bubblewrap	38
7.4.2. Configuración de Verificación y Requisitos Legales	38
7.4.3. Fase de Lanzamiento: Prueba Cerrada	38
8. Pruebas y Validación	41
8.1. Estrategia de pruebas técnicas	41
8.2. Pruebas con usuarios reales (User Testing)	41
8.3. Análisis de Resultados y Mejoras aplicadas	41
9. Conclusiones y Trabajo Futuro	43
Introduction	45
Conclusions and Future Work	47
Contribuciones Personales	49
Bibliografía	51
A. Título del Apéndice A	53
B. Título del Apéndice B	55

Índice de figuras

Índice de tablas

3.1. Configuración de una <i>Issue</i> en GitHub Projects.	15
--	----

Capítulo 1

Introducción

En este capítulo se introducen los conceptos básicos del proyecto, centrándonos en la escalada en rocódromo y en el uso de técnicas de gamificación aplicadas al deporte. Posteriormente se justifican los la motivación y objetivos de este preyecto y se resume la estructura de la memoria.

1.1. Contexto de la escalada en rocódromo

La escalada es un deporte con numerosas modalidades de las cuales nos centraremos en la "indoor" que se practica en rocódromos. A continuación se detalla su funcionamiento, tipos de ruta y el sistema de dificultad.

1.1.1. Funcionamiento de la escalada en rocódromo

Un rocódromo es una instalación deportiva que emula la escalada en roca natural mediante muros artificiales equipados con presas (agarres) de diferentes formas, tamaños y colores. El objetivo del escalador es ascender por una ruta predefinida usando exclusivamente las presas designadas a esta ruta.

Las rutas se identifican mediante un mismo color de presas. Cada ruta tiene un punto de inicio y un punto final o *top* (última presa que se debe agarrar con ambas manos para completar el ascenso). Cuando una ruta se completa en el primer intento, se denomina *flash*.

A diferencia de la escalada en roca natural, las rutas de los rocódromos son dinámicas. Con el objetivo de dar variedad y nuevos desafíos a los escaladores las rutas se van renovando periódicamente. Esta labor es llevada a cabo por los *route setter* (o equipadores), profesionales encargados de diseñar y montar los nuevos bloques y vías.

1.1.2. Tipos de rutas en rocódromos

Las principales modalidades practicadas en rocódromo son la escalada de bloque (*boulder*) y la escalada deportiva (vía):

1. **Escalada de bloque (búlder).** Se practica en muros de baja altura, normalmente de hasta 4 o 5 metros con colchonetas para la seguridad. El objetivo de este tipo de rutas es buscar una solución para problemas con secuencias de movimientos cortos e intensos.
2. **Escalada deportiva (vía).** Se realiza en muros de mayor altura por lo que se necesita material de seguridad como cuerda, arnés y sistema de aseguramiento. En estos tipos de ruta se requiere completar recorridos largos donde predominan la resistencia y la gestión del esfuerzo. Durante el ascenso, el escalador engancha la cuerda en anclajes intermedios y el asegurador, desde el suelo, controla la cuerda para detener posibles caídas.

1.1.3. Dificultad y progreso

Cada ruta, tanto de bloque como de vía, suele tener asignado un grado de dificultad que permite estimar el nivel exigido para completarlo. Este grado suele aparecer en la etiqueta de inicio.

En bloque se utilizan, entre otras, la escala de Fontainebleau (por ejemplo, 6A+, 7B) y la *V-Scale* (por ejemplo, V4, V6). En cambio en vía son frecuentes la escala francesa similar a Fontainebleau en notación y la YDS (por ejemplo, 5.10a, 5.11c). Una explicación detallada de estas escalas y sus conversiones puede consultarse en¹. Existen también análisis estadísticos de ascensos que aportan referencias de progresión, aunque no siempre son totalmente representativos².

Además de estas escalas, muchos rocódromos emplean sistemas internos basados en colores para asociar rangos de dificultad a cada ruta. Estos sistemas facilitan la lectura del nivel y pueden servir como elemento de motivación al permitir visualizar avances.

1.2. Contexto de la gamificación en deportes

La gamificación consiste en aplicar principios y componentes propios de los juegos en contextos no lúdicos, entrenamiento deportivo. El objetivo es incrementar la motivación, mejorar la retención de usuarios y favorecer la adherencia al deporte.

1.2.1. Modelo psicológico RAMP

Una de las bases principales para diseñar experiencias gamificadas es el modelo psicológico RAMP (*Relatedness, Autonomy, Mastery, Purpose*), alineado con teorías de motivación intrínseca. En este modelo se destacan cuatro necesidades principales de los usuarios:

- **Relatedness (vinculación):** necesidad del usuario de tener conexiones sociales, reconocimiento y pertenencia a una comunidad.

¹<https://www.lacrux.com/es/escalar/Escalas-de-escalada-explicadas-UIAA-Fontainebleau-V-Grado-CC>

²<https://www.alessandromasullo.com/blog/analysis-of-4-million-climbing-ascents/>

- **Autonomy (autonomía):** necesidad del usuario de sentir que el tiene el control y toma decisiones para progresar.
- **Mastery (maestría):** necesidad de sentir una mejora continua mediante objetivos alcanzables y progresivos.
- **Purpose (finalidad):** necesidad de sentir que el esfuerzo tenga un proposito más alla de la tarea inmediata.

Más información sobre motivación y gamificación puede consultarse en³.

1.2.2. El sistema PBL (*Points, Badges, Leaderboards*)

Basado en el modelo RAMP surge el sistema PBL. Este es una implementación clásica de gamificación y se compone de tres elementos principales:

- **Points (puntos):** representan el esfuerzo o rendimiento del usuario (por ejemplo, dificultad encadenada, número de intentos o constancia).
- **Badges (insignias):** representan logros alcanzados y visualizan la evolución del deportista (por ejemplo, completar 100 rutas o lograr tu primera ruta de dificultad 7a).
- **Leaderboards (clasificaciones):** comparan resultados entre usuarios, incentivando una competitividad sana y el sentido de comunidad.

Una explicación más detallada del modelo PBL se describe en⁴.

1.2.3. Beneficios de la gamificación en el deporte

La integración de mecánicas de gamificación en aplicaciones deportivas aporta beneficios relevantes:

- **Mayor retención:** la retroalimentación constante reduce el abandono y mejora la constancia del entrenamiento.
- **Visualización del progreso:** el registro de datos permite visualizar la evolución mediante métricas y logros.
- **Refuerzo de comunidad:** la interacción social aumentan la implicación de los usuarios con el deporte y su comunidad.

En un deporte individual como la escalada, donde el progreso puede ser lento y hay mucho estancamiento, estas mecánicas resultan especialmente útiles para mantener la motivación y constancia en el deporte.

³<https://playmotiv.com/gamificacion-y-motivacion/>

⁴https://www.adrformacion.com/knowledge/educacion-y-formacion/la_gamificacion_y_la_triada_pbl.html

1.3. Motivación

La escalada es un deporte unipersonal en el que el seguimiento del progreso suele realizarse de forma manual (libretas, memoria o aplicaciones externas no integradas en el propio rocódromo). La motivación principal de este proyecto es digitalizar el deporte en ciertos centros mediante una aplicación centralizada.

La aplicación permite registrar rutas completadas específicas de cada rocódromo, conservando el registro de actividad para analizar la evolución del usuario. Además, incorpora elementos de gamificación como estadísticas o Leaderboards para transformar el entrenamiento en una experiencia más social y motivadora, favoreciendo la constancia y una competitividad saludable entre la comunidad de escalada.

1.4. Objetivos

En esta sección se definen los objetivos que guían el desarrollo del proyecto, haciendo una distinción entre la meta global y los hitos técnicos necesarios.

1.4.1. Objetivo general

Desarrollar una aplicación compatible con dispositivos móviles que permita a los usuarios de un rocódromo registrar su actividad de escalada (bloque y vía) y que, mediante técnicas de gamificación, fomente la motivación y la constancia del entrenamiento.

1.4.2. Hitos específicos

1. Implementar un sistema de gestión de usuarios que cubra registro, autenticación y perfil personal con información de sus datos y estadísticas.
2. Desarrollar un sistema para escaladores de registro del estado de rutas que permita cambiar entre *flash*, completadas o proyecto.
3. Desarrollar un sistema de gestión de rutas para los *route setters* de manera que se reduzca al mínimo el esfuerzo necesario para registrar los cambios en el rocódromo.
4. Incorporar mecánicas de gamificación orientadas a mejorar la motivación del usuario.
5. Implementar funcionalidades sociales (amistades, comparación de progreso y estadísticas) para fomentar la competitividad sana en la comunidad.
6. Diseñar una interfaz y una experiencia de usuario simples, intuitivas y eficientes, preparadas para usuarios con distintos niveles tecnológicos.

1.5. Estructura del documento

Esta memoria se organiza en nueve capítulos que detallan el proceso completo de planificación, diseño y desarrollo de la aplicación. A continuación se resume brevemente el contenido de cada uno:

- **Capítulo 1: Introducción.** Presenta el contexto de la escalada *indoor*, los fundamentos de la gamificación (modelo RAMP y sistema PBL) y la motivación y los objetivos del trabajo.
- **Capítulo 2: Estado del Arte.** Realiza un análisis comparativo de aplicaciones de escalada y de otros deportes, identificando la oportunidad de mercado en la que se asienta esta propuesta.
- **Capítulo 3: Metodología de Desarrollo y Planificación.** Detalla el marco de trabajo Scrumban, las herramientas utilizadas y el rol de la inteligencia artificial como herramienta al desarrollo.
- **Capítulo 4: Análisis y Especificación de Requisitos.** Se definen los roles de usuario y las funcionalidades clave mediante historias de usuario.
- **Capítulo 5: Diseño del Sistema y Decisiones Técnicas.** Expone el *stack* tecnológico, la arquitectura del sistema (*backend* y *frontend*), el diseño de la base de datos y la interfaz de usuario.
- **Capítulo 6: Implementación y Desarrollo.** Se describe el proceso de desarrollo de los componentes software y los retos técnicos superados.
- **Capítulo 7: Integración Continua, Despliegue y DevOps.** Explica la configuración del *pipeline* de CI/CD, la gestión de infraestructura y el proceso de publicación en Google Play.
- **Capítulo 8: Pruebas y Validación.** Recoge la estrategia de las pruebas realizadas y los resultados de las sesiones con usuarios reales.
- **Capítulo 9: Conclusiones y Trabajo Futuro.** Se determinan los resultados obtenidos, se evalúa el cumplimiento de los objetivos definidos además de proponer una serie de tareas para el futuro del proyecto.

El documento finaliza con las contribuciones personales de los autores, la bibliografía consultada y los apéndices correspondientes.

Capítulo 2

Estado del Arte

Una vez determinado el objetivo general de este proyecto, continuaremos en este capítulo analizando el trasfondo y estado del arte en dos áreas clave: aplicaciones específicas de escalada y mecánicas de gamificación aplicadas en otros deportes.

Se presenta primero un análisis del panorama tecnológico actual examinando aplicaciones relevantes, sus funcionalidades y debilidades. Posteriormente se definen los principios de diseño y mecánicas de gamificación que se pueden extrapolar para el desarrollo de la aplicación.

2.1. Análisis de aplicaciones de escalada

En esta sección se analizan las principales aplicaciones que se emplean para registrar la actividad en la escalada. Las aplicaciones se han agrupado en base a su propósito definido.

2.1.1. Aplicaciones integradas con el rocódromo

Estas aplicaciones tienen una integración estrecha con los rocódromos, actuando como una herramienta para el registro de rutas y sesiones.

TopLogger (Top, 2026): TopLogger es una aplicación móvil que permite registrar el estado con las rutas de los rocódromos asociados y facilita la identificación de estas por ubicación, color y grado. Para determinar las posiciones de las rutas en un rocódromo realiza a través de un mapa cenital que permite moverte y seleccionar las rutas deseadas rápidamente. En el perfil se pueden visualizar numerosas estadísticas sobre datos de las sesiones de escalada y para su capa de gamificación tienen implementado un sistema de niveles y clasificaciones por rocódromo.

Entre sus limitaciones la primera que salta a la vista es la sobrecarga de información en cada pantalla, haciendo poco intuitiva y poco accesible la aplicación para perfiles con un nivel tecnológico bajo. En cuanto a la gamificación, al hacer las clasificaciones por rocódromo se puede engañar a la aplicación y posicionarte primero, consiguiendo el efecto adverso de motivación para los escaladores legítimos. Además

de esto, cuenta con poca integración con los rocódromos en España y con una capa de pago si tu rocodromo cuenta con muchos escaladores habituales.

WeClimb (WeC, 2026): WeClimb presenta funcionalidades similares a TopLogger, incluyendo un registro de rutas por color y grado pero difiriendo en la posición de las rutas. En esta aplicación para seleccionar la ruta usa un sistema de tarjetas de manera que seleccionando una sección de un mapa, se muestra una serie de tarjetas con las fotos de cada ruta facilitando el reconocimiento visual. En las imágenes de las rutas existe un mecanismo para poder resaltar las presas(agarres) que se pueden usar, dando la posibilidad de crear retos personalizados. Además de esto la aplicación cuenta con una pestaña para eventos, un historial con las tarjetas de las rutas completadas y clasificaciones por rocódromo.

Como limitaciones encontramos que su despliegue está centrado para Francia por lo que no hay centros ni traducción de la app. Actualmente carece de una capa de gamificación profunda, a excepción de las clasificaciones, pero que como en TopLogger se pueden engañar creando un efecto adverso. Por último el sistema de tarjetas para las rutas puede ser algo tedioso si hay numerosas rutas en una zona haciendo lento el proceso de selección de estas.

2.1.2. Aplicaciones centradas en la comunidad

En este grupo se prioriza la conexión entre escaladores frente al registro de todos los ascensos. Funcionan como redes sociales especializadas para compartir los ascensos más importantes.

KAYA(KAY, 2026) KAYA se posiciona como una red social para escaladores (especialmente orientada a escalada en roca natural), centrada en compartir ascensos con formato de publicaciones, interactuar con usuarios y seguir la progresión de otros. Las publicaciones pueden ser registros de una ruta, registro de una sesión de escalada o subir un video corto en formato vertical de una solución. En cuanto a elementos gamificación tiene un sistema de Badges en base al número de rutas registradas en la aplicación motivando a los escaladores a registrar más rutas.

Como limitaciones determinamos que la aplicación está más orientada al registro de rutas relevantes para el escalador más que el registro completo de toda su actividad. Además, la aplicación esta principalmente centrada en escalada en roca natural por lo que tiene poca integración con rocódromos.

2.1.3. Aplicaciones con hardware específico

Existen soluciones que combinan software con elementos físicos del rocódromo para crear experiencias de entrenamiento más interactivas.

MoonBoard(Moo, 2026) MoonBoard es una instalación física de entrenamiento que combina un muro de escalada con una aplicación. La particularidad de este sistema reside en que la posición de las presas y la inclinación del muro son idénticas en

todas sus instalaciones, permitiendo así una experiencia común en cualquier centro de escalada. La aplicación se conecta al muro y permite elegir entre miles de rutas que los escaladores han creado y una vez seleccionado uno, las presas que se pueden usar se iluminan. MoonBoard tiene un sistema de ranking global que funciona por un sistema de ascensos validados por la comunidad permitiendo a los usuarios competir con escaladores de otros países y visualizar el progreso en comparación con la media mundial.

Como limitaciones destacamos que MoonBoard es un sistema diseñado para entrenamientos de alta intensidad por lo que no es accesible a escaladores novatos. Además de esto MoonBoard es un sistema cerrado por lo que únicamente se pueden registrar las rutas completadas en el funcionando como una herramienta de registro parcial.

2.2. Análisis de aplicaciones de otros deportes

En esta sección se examinan mecánicas de gamificación probadas en otras disciplinas y aplicaciones de fitness con el objetivo de identificar ideas transferibles al contexto del rocódromo.

2.2.1. Competiciones y ranking por segmentos (Strava)

Strava(Str, 2026) Strava es una aplicación de registro de sesiones de corredores y ciclistas. Parte de su éxito viene de su concepto de segmentos, que son tramos específicos de una ruta donde los usuarios registran y comparan los tiempos que han tardado en completarlo mediante GPS. Este sistema transforma un entrenamiento individual en una competición virtual y el modelo se puede extrapolar a la escalada tratando cada vía o bloque como un segmento, donde se pueden medir el número de intentos o tiempo. En cuanto a la gamificación tiene un sistema de Badges que son títulos como KOM/QOM (King/Queen of the Mountain) que fomentan la motivación de los usuarios.

Como limitaciones en el enfoque de la escalada determinamos que un sistema de segmentos basados en tiempo en la escalada es contraproducente ya que se puede comprometer la técnica y seguridad del ascenso.

2.2.2. Coleccionismo de logros (Pokémon Go)

Pokémon Go(Pok, 2026) Pokémon Go es una aplicación del universo Pokémon centrada en la gamificación del desplazamiento físico con la vinculación de recompensas virtuales y el progreso en la narrativa de un juego. Este sistema permite convertir el esfuerzo en un incentivo para obtener el sentimiento de coleccionismo y descubrimiento. La aplicación es un ejemplo de la implementación del sistema PBL (Points Badges and Leaderboards) previamente definido.

La desventaja principal de esta aplicación es que el objetivo del usuario deja de ser el deporte y pasa a ser la recompensa del juego, creando una posible dependencia negativa en los usuarios.

2.3. Conclusiones del Estado del Arte e identificación de la oportunidad

Para finalizar este capítulo, se presentan las conclusiones obtenidas tras el análisis comparativo entre las aplicaciones, permitiendo identificar la necesidad del mercado y la propuesta de valor de nuestra aplicación.

2.3.1. Síntesis de la problemática actual

Tras analizar las soluciones existentes, se observa que el mercado de aplicaciones de escalada presenta problemas en tres aspectos críticos:

- **Experiencia de Usuario (UX) deficiente:** Aplicaciones como TopLogger o WeClimb ofrecen una gran densidad de datos, pero a costa de interfaces saturadas que dificultan el uso y registro de rutas en la aplicación. La barrera tecnológica aleja al escalador casual que solo busca un registro sencillo de su progresión.
- **Gamificación superficial:** La mayoría de las herramientas se limitan a clasificaciones numéricas (leaderboards) fácilmente manipulables, lo que genera desmotivación. Falta un sistema de recompensas basado en el esfuerzo, la constancia y el "coleccionismo" de hitos personales que no dependa únicamente de ser el mejor del ranking.
- **Dependencia de hardware o geografía:** Existen buenas soluciones como MoonBoard, pero están limitadas a sus muros de escalada específicos, o aplicaciones como WeClimb y TopLogger cuya implantación en España es inexistente. No existe una herramienta universal, intuitiva y "gamificada" que el escalador pueda llevar de un rocódromo a otro.

2.3.2. Identificación de la oportunidad

El análisis de aplicaciones como **Strava** y **Pokémon Go** revela que existe un gran potencial en la adaptación de mecánicas de gamificación de otros sectores a la escalada. La oportunidad reside en desarrollar una solución que combine:

1. **Registro visual y ágil:** Adoptar un sistema híbrido de mapas centiales dinámicos y sistema de tarjetas para minimizar el tiempo de interacción de escalador para reconocer y acceder a la ruta con el móvil durante la sesión.
2. **Gamificación basada en PBL:** Realizar una implementación correcta de el sistema PBL basado en el modelo RAMP de forma que las estadísticas funcionen como los *Points* para mantener la motivación y constancia de los usuarios, además de la implementación de Leaderboards entre grupos de amigos eliminando el factor de los abusos de los sistemas de posicionamiento.

3. **Enfoque social y comunitario:** Fomentar la interacción mediante personalización del perfil y la comparación de estadísticas personales, creando un sentimiento de pertenencia a la comunidad del rocódromo local.

En conclusión, este proyecto se posiciona entre las herramientas de entrenamiento puro y las redes sociales generales, ofreciendo un sistema de entrenamiento que registra datos de las sesiones y motiva activamente al escalador a través de una experiencia lúdica y bien diseñada.

Capítulo 3

Metodología de Desarrollo y Planificación

Uno de los factores más importantes en el desarrollo de un proyecto es el plan y metodología que se va a llevar a cabo para su desarrollo. En este capítulo detallamos cómo se ha definido la metodología de nuestra aplicación *ClimbIt* para lograr un flujo de trabajo funcional con sus herramientas específicas para la gestión y organización que permite ir avanzar eficientemente en las diferentes fases del proyecto.

3.1. Metodología de trabajo: Scrumban

Como metodología de trabajo nos decantamos por utilizar **Scrumban** debido a que consideramos que aplicar Scrum puro podía resultar demasiado estricto especialmente para un equipo de dos personas y en Kanban echamos en falta periodos de trabajo intensos y acotados en el tiempo. Scrumban es una metodología híbrida que combina lo mejor de ambos mundos, permitiendo la flexibilidad de Kanban con la estructura de Scrum.

De Scrum nos quedamos con el concepto de *sprints* (iteraciones temporales de desarrollo) de dos semanas aproximadamente y de las técnicas de especificación de requisitos mediante *Historias de usuario* y *Epics* (estructuras para la definición de casos de uso). Por otro lado, de Kanban en lugar de preasignar tareas de forma estricta trabajamos con un sistema *pull* de forma que cada miembro del equipo selecciona la tarea a realizar de la columna *To Do*.

3.2. Proceso de Captura de Requisitos y Definición del Producto

Antes de iniciar la fase de desarrollo e implementación, determinamos que era fundamental comprender en profundidad los problemas a resolver, de manera que no cometiéramos el error de basar la programación en suposiciones no contrastadas. Por lo que dedicamos una fase inicial a la definición estratégica del producto, apoyándonos en herramientas de diseño ágil.

3.2.1. Lienzo de Propuesta de Valor (Value Proposition Canvas)

Con el objetivo de empatizar de manera efectiva con el usuario final, construimos un Value Proposition Canvas, siendo esta una herramienta que nos permite adoptar la perspectiva del escalador y analizar su contexto dividiéndolo en dos bloques principales, el Perfil de cliente y Mapa de valor.

Perfil del cliente : Se enfoca exclusivamente en entender al usuario. Te pones en su situación y describes sus *trabajos* (lo que intenta resolver), sus *frustraciones* (los obstáculos y emociones negativas que encuentra) y sus *ganancias* (los resultados y beneficios que desea obtener).

Se identificó que el usuario escalador busca cumplir trabajos funcionales (registrar sesiones, ver pistas), sociales (compartir logros) y emocionales (sentirse motivado). Sus principales frustraciones giran en torno a la dificultad del registro manual, la falta de una plataforma centralizada y la desmotivación por no visualizar su progreso. Por otro lado, sus ganancias deseadas son la facilidad para seguir su evolución, recibir feedback y sentirse parte de una comunidad competitiva y sana.

Mapa de Valor Se enfoca en cómo un producto o servicio crea valor en base a los datos del perfil de cliente. Aquí se describen los *productos y servicios* alivian frustraciones de tus clientes (*Pain Relievers*) y cómo generan las ganancias que ellos buscan (*Gain creators*).

Para responder al perfil del cliente, se determinó que los *Pain Relievers* se enfocan en ofrecer una interfaz intuitiva, un registro de progreso automatizado y una plataforma segura y centralizada. Finalmente, sus *Gain Creators* se centran en proporcionar estadísticas detalladas, un sistema de logros motivador y una comparativa social que impulse al usuario a mejorar e interactuar con sus compañeros.

3.2.2. Visualización del viaje: User Story Map

A partir de la propuesta de valor, se construyó un User Story Map para visualizar el recorrido completo del usuario dentro de la aplicación, utilizando la plataforma Mural. Un USR (User Story Map) es una técnica visual utilizada en la gestión ágil de proyectos para organizar y priorizar las historias de usuario. Este enfoque facilita la comprensión del flujo de trabajo y las interacciones del usuario con el sistema, ayudando a definir y planificar las funcionalidades del producto de manera efectiva.

Este mapa visual resultó muy importante ya que nos ayudó a dividir el proyecto. Trazando líneas horizontales sobre el mapa, pudimos decidir qué funcionalidades eran vitales para el primer prototipo, cuáles entrarían en el Producto Mínimo Viable (MVP) y qué ideas quedaban relegadas a versiones futuras.

3.3. Herramientas de gestión y seguimiento

Para la gestión del proyecto tomamos la decisión de tener las herramientas lo más cercanas posibles para reducir el rozamiento de su uso. Descartamos usar plataformas externas tipo Jira o Trello y centralizamos toda la gestión directamente en **GitHub Projects**, manteniendo los requisitos vinculados al código fuente y las herramientas de despliegue.

El *Product Backlog* se fue rellenando progresivamente con las tareas extraídas del User Story Map. Cada requisito, ya fuera una Historia de Usuario (HU) funcional o una tarea técnica (como configurar la base de datos), se tradujo en una *Issue* dentro del repositorio con información suficiente para no perder contexto.

Atributo	Descripción y uso en el proyecto
Estado	Columna del tablero Kanban (<i>Backlog</i> , <i>To Do</i> , <i>In Progress</i> , <i>Review</i> , <i>Done</i>).
Labels	Categorización visual (<i>bug</i> , <i>backend</i> , <i>frontend</i> , <i>documentación</i> , <i>epic</i> , <i>HU</i> ...).
Estimación	Tamaño (<i>Size</i>) desde XS hasta XL para medir el esfuerzo relativo.
Prioridad	Nivel de urgencia (<i>P0</i> , <i>P1</i> , <i>P2</i>), marcando P0 las tareas críticas de bloqueo.
Milestone	Vinculación directa al <i>sprint</i> actual para seguimiento temporal.
Criterios	Listado de condiciones exactas para considerar la tarea como finalizada.

Tabla 3.1: Configuración de una *Issue* en GitHub Projects.

Además, el seguimiento de los cuatro *sprints* se apoyó en un *Roadmap* visual y en gráficos de *Burndown*, que nos permitía visualizar el progreso y detectar si habíamos sobrestimado nuestra capacidad de desarrollo.

3.4. Gestión de la Configuración y Control de Versiones

Implementamos un flujo de trabajo basado en **Git Flow**, estableciendo reglas claras sobre cómo se integra el trabajo de cada desarrollador.

En el repositorio existen dos ramas principales protegidas. La rama **main** se reservó exclusivamente para las versiones estables y liberadas del producto de manera que todo el código de esta rama se subía a producción, mientras que **develop** actuaba como el entorno de preproducción para los nuevos casos de uso. Cada *Issue* del tablero requería la creación de una rama específica (*feature branch*). Al finalizar el desarrollo, esta rama se integraba en **develop** mediante una estrategia de *squash and merge*. Esto nos permitió agrupar decenas de pequeños *commits* en un solo bloque semántico, manteniendo el historial del proyecto limpio y legible.

En paralelo a los ciclos de *sprint*, el despliegue de versiones se gestionó a través de la herramienta de *Releases* de GitHub. Adoptamos el versionado semántico clásico (X.Y.Z) y mantenemos la trazabilidad de los cambios mediante un archivo `CHANGELOG.md` que sigue los estándares de *Keep a Changelog*. Además para mantener la buena praxis el repositorio cuenta desde el inicio con archivos `.gitignore` configurados para el *stack* tecnológico y una guía de instalación en el `README.md`.

3.5. Uso de Inteligencia Artificial como Apoyo al Desarrollo

Durante el ciclo de vida del proyecto, hemos empleado herramientas de Inteligencia Artificial (IA) generativa como asistentes de desarrollo. El uso de esta tecnología no ha sustituido en ningún momento nuestra toma de decisiones, el diseño de la arquitectura o la lógica de negocio, sino que ha actuado como un recurso de apoyo para optimizar nuestros tiempos y aumentar el alcance del proyecto sin comprometer la calidad de este.

La integración de la IA en nuestro flujo de trabajo se ha estructurado de la siguiente manera:

Investigación y adopción de buenas prácticas: En las fases de exploración técnica, la IA se utilizó como un motor de búsqueda. Ante la necesidad de implementar nuevas funcionalidades o integrar herramientas desconocidas, recurrimos a ella para evaluar diferentes alternativas, comprender sus enfoques y asimilar buenas prácticas. Esto nos permitió filtrar grandes volúmenes de documentación de manera eficiente, facilitando una toma de decisiones más informada y acelerando la curva de aprendizaje.

Automatización de tareas repetitivas y reducción de carga cognitiva: En el desarrollo del código de nuestro proyecto, nos hemos encontrado con muchas fases de repetición de código repetitivo que no aportan valor innovador. Nuestra norma de uso consistió en que, una vez que la arquitectura, el diseño o la implementación de una funcionalidad base habían sido definidos, validados y programados manualmente al menos una vez por el equipo, utilizábamos la IA para replicar y adaptar estos patrones en otros componentes similares. Al delegar este trabajo de repetición mecánica, pudimos redirigir nuestra concentración hacia tareas críticas, dedicando mucho más tiempo y esfuerzo a la revisión exhaustiva, el control de calidad y la optimización del código final.

Capítulo 4

Análisis y Especificación de Requisitos

En este capítulo se detalla a quién va dirigida la aplicación junto con los roles que intervienen y las funcionalidades y restricciones que debe cubrir. Para ello, primero se definen los perfiles de usuario clave (*Personas*) para entender sus motivaciones y necesidades. A continuación, definimos los requisitos funcionales y no funcionales. Por último, se establece el alcance de cada versión del proyecto.

4.1. Personas

A continuación, se presentan dos perfiles de usuario o *Personas*, que representan los dos roles principales de la aplicación: el escalador y gestor del rocódromo.

4.1.1. Persona 1: Klaudia, Escaladora Constante

Se presenta de un perfil ficticio para representar a una escaladora que frecuenta el rocódromo y quiere registrar su progreso y mantenerse motivada.

- **Perfil:** Klaudia, Escaladora habitual, 24 años.
- **Nivel de escalada:** Intermedio (6a - 6c).
- **Frecuencia:** Va al rocódromo 2-3 veces por semana.
- **Biografía:** Klaudia se inició en la escalada hace un par de años y se ha convertido en su principal pasatiempo. No se considera una profesional, pero se toma en serio su progreso y va al rocódromo varias veces por semana, con su grupo de amigos o si hace falta sola. Conoce bien las instalaciones y se da cuenta de que las rutas de los muros van cambiando cada semana. Le encanta la sensación de completar bloques por encima de su nivel, pero le frustra no ver de forma clara su progreso.
- **Metas y Motivaciones:**
 - Quiere poder visualizar las rutas que ha completado para ver su progreso.

- Disfruta del componente social de la escalada y le gusta picarse con sus amigos.
 - Busca una forma de poder competir con sus amigos para esforzarse más en cada sesión de escalada.
- **Frustraciones y Puntos de Dolor:**
- Le resulta molesto tener que llevar un registro de sus ascensos en una libreta o de memoria.
 - A menudo se olvida qué rutas ha completado en sesiones anteriores.
 - Se siente estancada porque no tiene datos para analizar sus puntos fuertes y débiles.
 - No existe una forma fácil de compararse con sus amigos.
- **Relación con la aplicación (*ClimbIt*):** Klaudia es un usuario ideal para la aplicación ya que la usaría en cada sesión para registrar sus rutas completadas, seguir las estadísticas de su perfil y participar en los rankings. La gamificación y los retos sociales serían una fuente de motivación para dar más de sí misma.

4.1.2. Persona 2: Victor, Gestor del Rocódromo

Se presenta el perfil para representar al gestor de las instalaciones que busca satisfacer a los escaladores y gestionar eficientemente su muro.

- **Perfil:** Victor, *Route-Setter* principal, 42 años.
- **Rol:** Responsable de la renovación de las rutas.
- **Biografía:** Victor es el dueño del rocódromo y lleva más de una década gestionando el centro y es el principal encargado de diseñar y montar las nuevas rutas. Su objetivo es mantener el rocódromo vivo y desafiante para escaladores de todos los niveles de manera que cada semana, cambia una sección del muro para que los escaladores habituales siempre encuentren nuevos retos.
- **Metas y Motivaciones:**
- Quiere ofrecer la mejor experiencia posible a todos los escaladores de su rocódromo.
 - Le interesa recibir *feedback* de los usuarios sobre la dificultad y calidad de sus rutas.
 - Quiere crear una comunidad unida en su centro para que los escaladores quieran volver y recomendarlo a otros.
- **Frustraciones y Puntos de Dolor:**
- No tiene una forma de saber qué rutas son las más populares o cuáles no han gustado.

- Le gustaría tener una forma más directa para interactuar con la comunidad de su rocódromo.
 - Gestionar cientos de rutas activas puede ser un proceso manual y desorganizado.
- **Relación con la aplicación (*ClimbIt*):** Victor podrá utilizar el panel de gestión de la aplicación para gestionar todas las rutas del rocódromo. Para él, la app es una herramienta de gestión y marketing que le permite mantener a los escaladores informados, obtener datos valiosos sobre sus rutas y potenciar la comunidad, diferenciando a su rocódromo de la competencia.

4.2. Roles

Para gestionar todas las funcionalidades de la aplicación, se ha definido un modelo de permisos basado en cuatro roles principales. Cada rol cuenta con un nivel de privilegios y funcionalidades específicas dentro de la plataforma:

4.2.1. Escalador

Es el rol del usuario final de la aplicación. Su objetivo principal es registrar sus estados con las rutas y participar en la comunidad. Tiene permisos de lectura sobre la información pública de los rocódromos y permisos de escritura sobre su información. Sus funciones clave incluyen son la gestión de su perfil personal, la interacción social con amigos, el registro de sus ascensos y la consulta de estadísticas y clasificaciones (*leaderboards*).

4.2.2. Gestor de Rocódromo

Se trata de un rol organizativo asignado a los trabajadores o *route-setters* de un centro de escalada específico. Tienen además de los permisos de un escalador, los de gestión de rutas de sus rocódromos y sus funciones principales abarcan la gestión de las rutas (crear, modificar, actualizar estados y eliminar las rutas disponibles).

4.2.3. Propietario

Es el administrador principal de un centro de escalada en la plataforma y hereda todos los permisos del rol de Gestor de Rocódromo (gestión de rutas), pero cuenta además con permisos de administración sobre el rocódromo. Puede modificar la información del centro, como la ubicación, los horarios de apertura y el logotipo.

4.2.4. Administrador

Es el rol de máximo nivel reservado para los desarrolladores (*Superusuario*). Cuenta con acceso total y control absoluto sobre la plataforma donde se puede gestionar, crear, modificar y eliminar cualquier entidad de la aplicación, incluyendo usuarios, zonas, rocódromo y rutas.

4.3. Especificación de funcionalidades (Historias de Usuario)

Las funcionalidades que se han implementado se han obtenido del *Product Backlog* y se han listado agrupadas según la temática dentro del proyecto. Unicamente se muestra el identificador y su título. Para más detalles se puede consultar en el apéndice A.

4.3.1. Autenticación y Gestión de Perfil

Historias de usuario relacionadas con el acceso al sistema y la personalización de la cuenta del escalador:

- **CI-1:** Iniciar sesión con correo y contraseña.
- **CI-2:** Cerrar sesión.
- **CI-10:** Registro con correo electrónico.
- **CI-47:** Tutorial al registrarse (*Onboarding*).
- **CI-3:** Mostrar información básica del perfil.
- **CI-21:** Modificar descripción del perfil.
- **CI-22:** Cambiar foto de perfil entre predefinidas.
- **CI-23:** Mostrar estadísticas básicas del perfil.
- **CI-24:** Cambiar apodo.
- **CI-25:** Mostrar descripción del perfil.
- **CI-45:** Estadísticas de escalador en rocódromo específico.

4.3.2. Social y Comunidad

Historias de usuario centradas en la interacción entre los usuarios :

- **CI-13:** Buscar usuario por apodo.
- **CI-14:** Enviar Solicitud de Amistad.
- **CI-15:** Gestión de Solicitudes Recibidas.
- **CI-16:** Mostrar listado de Amigos.
- **CI-17:** Eliminar Amigo.
- **CI-44:** Ver perfil de amigo.

- **CI-46:** *Leaderboard* de rutas del mes actual escaladas entre amigos.
- **CI-28:** Crear grupo de escaladores.
- **CI-29:** Enviar Invitación a grupo.
- **CI-30:** Gestionar invitación de unión a grupo.
- **CI-32:** Visualizar estadísticas de miembros de grupo.

4.3.3. Rocódromos

Historias de usuario orientadas a la exploración de centros por parte del escalador y a la administración general de los rocódromos por parte del gestor:

- **CI-8:** Escalador se suscribe a Rocódromo.
- **CI-9:** Escalador se desuscribe de Rocódromo.
- **CI-11:** Mostrar listado de “Mis rocódromos”.
- **CI-12:** Buscador de rocódromos básico.
- **CI-18:** Modificar información básica de rocódromo.
- **CI-26:** Añadir tipos de dificultad del rocódromo.
- **CI-37:** Mostrar Información básica de Rocódromo.

4.3.4. Zonas

Historias de usuario relacionadas con la visualización y gestión de los diferentes zonas dentro de un rocódromo:

- **CI-4:** Mostrar Mapa de Zonas de Rocódromo básico.
- **CI-38:** Mostrar Zonas con porcentaje de completado.
- **CI-39:** Mostrar mapa dinámico de zona.

4.3.5. Rutas

Historias de usuario relacionadas con la interacción con las rutas de escalada, tanto para el escalador como para el gestor del rocódromo:

- **CI-5:** Mostrar Mapa de Rutas de Zona básico.
- **CI-6:** Mostrar Rutas básico.
- **CI-7:** Cambiar estado de Ruta.
- **CI-19:** Añadir ruta al rocódromo.

- **CI-20:** Modificar una ruta.
- **CI-27:** Eliminar una ruta del rocódromo.
- **CI-40:** Añadir filtros de búsqueda por dificultad en zona.
- **CI-41:** Mostrar información avanzada de ruta.
- **CI-42:** Valorar ruta en formato de estrellas.
- **CI-43:** Porcentaje Acotado por dificultad.

4.4. Requisitos no funcionales

Además de las requisitos funcionales, el sistema debe cumplir una serie de requisitos no funcionales que garantizan la viabilidad, rendimiento y una correcta experiencia de usuario:

- **Usabilidad:** La interfaz debe ser altamente intuitiva, permitiendo que un escalador registrar rutas en poco tiempo para reducir la fricción con el dispositivo móvil durante su sesión de escalada.
- **Rendimiento y Eficiencia:** La carga de los mapas de las zonas del rocódromo y las listas de rutas debe ser fluida para evitar frustraciones.
- **Seguridad y Privacidad:** Debe haber un sistema de autenticación robusto y garantizar la correcta separación de datos para proteger la información personal de los escaladores.
- **Compatibilidad:** La aplicación debe ofrecer un diseño *responsive* orientado a dispositivos móviles (*Mobile First*), asegurando la correcta visualización en diferentes tamaños de pantalla y sistemas operativos.
- **Mantenibilidad:** El código debe estructurarse siguiendo patrones establecidos y siguiendo buenas prácticas, para facilitar futuras modificaciones y corrección de errores.

4.5. Alcance de las versiones

En este apartado se definen las funcionalidades implementadas en cada versión del proyecto, reflejando la evolución de la aplicación desde el prototipo hasta su estado actual.

4.5.1. Requisitos del prototipo (v.0.1.0)

Esta fue la primera versión y se estableció la base funciona del sistema, permitiendo un flujo básico para el escalador, junto con la preparación de la infraestructura técnica:

- **Autenticación y Perfil:** Implementación del inicio y cierre de sesión, registro mediante correo electrónico y visualización de la información básica del usuario.
- **Gestión de Rocódromos:** Desarrollo de un buscador de rocódromos, gestión de suscripciones (suscribirse y desuscribirse), y navegación básica por las zonas.
- **Interacción con Rutas:** Funcionalidad de listar rutas por zona, consultar su información detallada y actualizar el estado de escalada (flash, completado y proyecto).
- **Infraestructura Técnica:** Configuración de un entorno mediante contenedores (Docker), despliegue continuo (CI/CD) con GitHub Actions y creación de entornos para preproducción y producción.

4.5.2. Versiones intermedias (v.0.2.0 - v.0.2.2)

Durante estas iteraciones intermedias, el desarrollo se centró implementar funcionalidades para el rol de Gestor de Rocódromo, enriquecer la experiencia de usuario y optimizar el rendimiento en dispositivos móviles:

- **Personalización y Estadísticas:** Se implemento la edición avanzada del perfil (apodo, descripción y foto de perfil entre predefinidas). Además, se implementó un sistema de visualización de estadísticas que incluye el resumen global, actividad mensual con mapas de calor y estadísticas de distribución por tipos de ruta y dificultad.
- **Herramientas para Gestores:** Se desarrollo la capacidad para subir y actualizar la información del rocódromo, así como definir sus escalas de dificultad.
- **Gestión Dinámica de Rutas:** Se diseñaron y desarrollaron mapas dinámicos interactivos por zona. Para las rutas, se completaron las funcionalidades de creación, edición, borrado lógico, y un sistema de valoración por estrellas.
- **Optimización Móvil (PWA):** Se configuró la aplicación como *Progressive Web App* (PWA) instalable, optimizando la carga de imágenes e implementando modos de lectura sin conexión.

4.5.3. Requisitos del MVP (v.1.0.0)

El Producto Mínimo Viable (MVP) se completo añadiendo el factor social, y refactorizando el sistema para garantizar la accesibilidad en entornos de baja conectividad:

- **Funcionalidades Sociales:** Incorporación de un sistema de amistades que permite buscar usuarios, enviar o recibir solicitudes y gestionar la lista de amigos. Se habilitó la visualización de perfiles públicos y se integró un *Leaderboard* (tabla de clasificación) basado en rutas completadas y días activos en el mes.
- **Usabilidad y Retención:** Implementación de un tutorial interactivo de *onboarding* tras el registro o en el menú del perfil, validación en tiempo real para correos y apodos en el registro, y visualización de las imágenes de las rutas a pantalla completa.
- **Arquitectura *Offline-First*:** Reestructuración del *cacheo* de la aplicación para priorizar el funcionamiento sin conexión.

4.5.4. Mantenimiento y mejoras post-lanzamiento (v.1.0.1)

Tras el despliegue del MVP, se publicó un parche centrado para arreglar errores, añadir política de privacidad y mejorar detalles específicos de la interfaz:

- **Correcciones Funcionales:** Solución de fallo en la interfaz de mapas dinámicos en dispositivos iOS y reparación del fallo que impedía actualización de las listas de amigos y solicitudes de amistad.
- **Transparencia:** Añadida de una sección de Política de Privacidad accesible desde el perfil del usuario.
- **Ajustes Visuales:** Inclusión de nuevos colores para el etiquetado de rutas (azul celeste) y rediseño en las tarjetas del *Leaderboard* para una visualización más clara.

Capítulo 5

Diseño del Sistema y Decisiones Técnicas

En este capítulo se detallan las decisiones técnicas adoptadas para la implementación de Climbit así como la justificación de dichas elecciones frente a las alternativas actuales del mercado. También se detallarán las arquitecturas del frontend y del backend y los patrones de diseño utilizados.

5.1. Decisiones de Diseño y Stack Tecnológico

Para Climbit surgieron varias opciones de diseño para el sistema, pero al final optamos por una separación de responsabilidades para ceder la lógica de negocio y la persistencia de los datos al backend y un frontend SPA (*Single Page Application*) simple y de fácil navegación que consuma esos datos a través de una API. Desde el comienzo sabíamos que el alcance de esta aplicación era notablemente amplio, por lo que la decisión sobre qué arquitectura utilizar fue previamente meditada. Valoramos tres opciones de diseño para el backend: una arquitectura monolítica, una arquitectura hexagonal y una arquitectura de microservicios.

Para la elección tuvimos en cuenta tres factores principales: la escalabilidad, la mantenibilidad y la complejidad de desarrollo. Una arquitectura monolítica podría haber sido más sencilla de implementar inicialmente, pero a medida que el proyecto creciese, podría habernos llevado a un código difícil de mantener y escalar. Por otro lado, una arquitectura de microservicios nos habría ofrecido grandes ventajas en cuanto a la escalabilidad, pero la curva de aprendizaje y la dificultad que añadía al desarrollo y la necesidad de gestionar múltiples servicios nos hizo descartarla, por el momento, para este proyecto. Al final decidimos que la arquitectura hexagonal nos daba el equilibrio entre dificultad de desarrollo, mantenibilidad y escalabilidad que buscábamos para Climbit, permitiéndonos desacoplar toda la lógica de negocio de factores externos como la base de datos y el frontend, quedando así un sistema más modular y fácil de mantener a largo plazo.

En cuanto al stack tecnológico, para el backend optamos por usar Node.js con Express debido a su popularidad, fiabilidad, facilidad de uso y su amplio ecosistema de librerías. Elegimos trabajar directamente con JavaScript y en formato ESModules y no con TypeScript principalmente por la velocidad de desarrollo y porque Node.js no tiene un soporte nativo para TypeScript. Para la base de datos, descartamos des-

de el principio las bases de datos no relacionales, ya que la naturaleza de los datos de Climbit, con relaciones claras entre escaladores, rocódromos y rutas se adaptaba mejor a un modelo relacional. Ambos habíamos trabajado previamente con bases de datos como MariaDB y MySQL, pero debido a su rendimiento, escalabilidad y su más que creciente popularidad y ecosistema decidimos que para nuestro proyecto, y para las ideas que queremos llegar a implementar en el futuro, PostgreSQL era la mejor opción. Para la comunicación entre el backend y la base de datos, optamos por usar un ORM (*Object-Relational Mapping*) para facilitar el desarrollo, así como una capa extra de seguridad. Después de evaluar varias opciones elegimos que Sequelize, debido a su diseño nativo para JavaScript, era la opción más indicada. Otros ORMs como TypeORM o Prisma también eran opciones válidas, pero al estar más orientados a TypeScript no podríamos aprovechar al máximo sus características, por lo que Sequelize era la mejor opción para nuestro proyecto.

A la hora de servir imágenes al frontend, llegamos a la conclusión de que tener imágenes sin procesar en el servidor jugaría en nuestra contra a la hora de tener un buen rendimiento, por lo que decidimos generalizar el uso del formato webp en todas las imágenes que se subieran a la aplicación, exceptuando los mapas de las zonas, que se mantendrían en formato svg. Obligar a que los usuarios subieran las imágenes en un único formato generaría una gran fricción a la hora de usar la app; la solución más óptima era permitir una amplia gama de formatos para las imágenes y convertirlas a webp en el servidor antes de guardarlas. Para ello decidimos usar la librería ffmpeg, una de las librerías más populares y potentes para el procesamiento de imágenes que ya habíamos utilizado previamente en otro proyecto.

En cuanto a la gestión de sesiones, optamos por JWT (*JSON Web Tokens*) por su ligereza y capacidad de escala futura. Los tokens contienen la información necesaria sobre permisos e identidad del usuario, eliminando la necesidad de realizar validaciones completas con cada petición y mejorando así los tiempos de respuesta y el rendimiento. Además, al estar firmados con una clave secreta, garantizamos que los datos no pueden ser manipulados, proporcionando una capa adicional de seguridad sin sobrecargar el servidor con sesiones almacenadas.

Para el frontend sabíamos que queríamos una aplicación web, pero que fuera PWA (*Progressive Web App*), para que los usuarios de cualquier dispositivo pudieran acceder a ella y descargarla sin la necesidad de tener que crear una aplicación nativa para cada plataforma. Desde el principio fuimos reticentes con el uso de frameworks frontend. Después de valorar las ventajas que nos ofrecían frente a la curva de aprendizaje y la complejidad añadida que suponían, decidimos optar por una solución más ligera y sencilla: utilizar HTML, CSS y JavaScript puro, ayudados de Bootstrap para mantener un diseño consistente. Esto nos permitió tener un control total sobre el código y una velocidad inicial de desarrollo mucho mayor, ya que los dos habíamos desarrollado previamente con estas tecnologías. Por otro lado, sí nos supuso un reto la implementación de ciertas funcionalidades, en concreto el enrutamiento de las vistas o la implementación manual de componentes personalizados, que es algo que los frameworks manejan de forma nativa y que nosotros tuvimos que implementar desde cero, pero al final creemos que fue la decisión correcta para el proyecto.

5.2. Diseño de la Arquitectura

5.2.1. Backend

Una vez elegida la arquitectura hexagonal para el backend, se diseñó la estructura del proyecto siguiendo los principios de esta arquitectura y aplicando una serie de patrones de diseño que nos permitieran mantener un código limpio, sencillo y modular, completamente independiente entre las diferentes capas. La arquitectura hexagonal organiza el código en capas definidas, donde cada una tiene una responsabilidad clara. La capa de dominio se encarga de la lógica de negocio y las reglas del sistema, donde se definen todos los contratos y validaciones sobre los objetos que se procesan. La capa de infraestructura se encarga de la comunicación con la base de datos, que nosotros gestionamos a través de Sequelize, así como los modelos que representan las entidades del sistema, las migraciones y las seeds. La capa de aplicación se encarga de definir la lógica de los use cases, orquestando la interacción entre la capa de dominio y la capa de infraestructura. Por último, la capa de presentación, que nosotros hemos llamado interfaz, se encarga de la comunicación con el frontend a través de una API REST, definiendo los endpoints, los controladores que gestionan las peticiones y respuestas y los middlewares que nos ayudan a la autenticación de tokens, formatos de datos y validaciones tanto de datos como de archivos.

Diagrama de la arquitectura hexagonal del backend con ejemplo de Climbit

A lo largo del desarrollo nos encontramos con diversas situaciones que nos llevaron a aplicar diferentes patrones de diseño que nos ayudaron a obtener un código más limpio, modular y fácil de mantener. Los patrones de diseño que hemos utilizado son los siguientes:

Patrón Repository: Este patrón propuesto por Martin Fowler consiste en crear una interfaz entre la lógica de negocio y la capa de persistencia, en nuestro caso, nuestra base de datos PostgreSQL a través de Sequelize, de manera que nos permite desacoplar el negocio de la infraestructura, manteniendo así un código más mantenible en el tiempo. En nuestro caso, como se puede ver en la Figura XX, la capa de dominio define la interfaz del repositorio en la que se establecen los métodos que se necesitan para realizar las diferentes operaciones y la capa de infraestructura implementa dicha interfaz con la que hablar con la base de datos. De esta manera, la capa de aplicación solo se encarga de definir la lógica de los distintos casos de uso.

Figura del diagrama de patrón repository

Patrón Adapter: El patrón Adapter es uno de los patrones clásicos introducidos por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, también conocidos como los “Gang of Four” y se encarga de adaptar distintas interfaces para que puedan trabajar entre ellas. En nuestro caso, lo hemos utilizado para adaptar distintos errores que pueden surgir en las diferentes capas, como se puede observar en la Figura XX, para centralizar el manejo de estos errores en un único lugar, permitiéndonos así mantener una consistencia a la hora de tratar los distintos errores que pueden surgir.

Figura del diagrama de patrón adapter

Patrón Singleton: Introducido también por “The Gang of Four” en su libro “Design Patterns: Elements of Reusable Object-Oriented Software”, el patrón Singleton sirve para garantizar que una clase tenga únicamente una instancia al mismo tiempo a lo largo de la ejecución de la aplicación. En nuestro caso, lo hemos utilizado, por ejemplo, para mantener una única conexión a la base de datos, evitando así posibles problemas y errores relacionados con múltiples conexiones abiertas, pero también lo hemos utilizado en otros puntos que detallaremos más adelante en la implementación de la aplicación.

Inyección de dependencias e Inversión de control: La inversión de control es un principio abstracto popularizado por Robert C. Martin, también conocido como “Uncle Bob”, pero ideada inicialmente por Martin Fowler, que viene a referir que los objetos no deben crear sus propias dependencias, sino que le tienen que venir dadas de fuera. Por otro lado, la inyección de dependencias es una de las técnicas para implementar este principio. En nuestra aplicación hemos utilizado la inyección de dependencias a través de la inyección por constructor transversalmente en toda la aplicación. El controlador recibe por inyección los casos de uso que necesita para gestionar las peticiones, los use cases reciben por inyección los repositorios que necesitan para realizar sus operaciones y los repositorios reciben por inyección los modelos que necesitan para hablar con la base de datos. De esta manera, hemos conseguido un código completamente desacoplado alineado con la filosofía de la arquitectura hexagonal.

5.2.2. Frontend

Se describe cómo se organiza la interfaz de usuario y su comunicación con el backend.

5.3. Diseño de la Base de Datos

Como se mencionó anteriormente en la sección de decisiones tecnológicas, la naturaleza de los datos en ClimbIt presentaba relaciones claras y bien definidas entre entidades como escaladores, rocódromos, zonas y rutas, lo que nos llevó a descartar desde el principio las bases de datos no relacionales en favor de un modelo relacional. La estructura jerárquica y las múltiples relaciones muchos-a-muchos entre entidades hacían que un modelo relacional fuese la opción más apropiada para garantizar integridad referencial, coherencia de datos y facilidad de consultas complejas.

Elegimos PostgreSQL como sistema gestor de bases de datos relacional (SGBDR) debido a su robustez, rendimiento, escalabilidad y su creciente popularidad en el ecosistema de desarrollo. Para facilitar la interacción entre la aplicación y la base de datos, utilizamos Sequelize como ORM, que nos permite trabajar con los modelos de datos de forma orientada a objetos sin necesidad de escribir SQL directamente,

proporcionando además una capa adicional de seguridad contra inyecciones SQL y abstrayendo las particularidades específicas de PostgreSQL.

5.3.1. Entidades Principales del Sistema

El modelo de datos de Climbit está compuesto por las siguientes entidades principales:

- **Escalador:** Representa a los usuarios del sistema, almacenando información de perfil, credenciales de autenticación y configuración personal.
- **Rocódromo:** Entidad que representa cada uno de los lugares donde se practica la escalada, incluyendo información como nombre, ubicación y datos de contacto.
- **Zona:** Las diferentes áreas o sectores dentro de un rocódromo, cada una con su propio mapa y características.
- **Pista:** Las rutas específicas de escalada dentro de una zona, con información de dificultad, tipo de escalada y estado.
- **Ascentso:** Registro de cada vez que un escalador asciende una pista, almacenando información como fecha, valoración y estado del ascenso.
- **EscalaDificultad:** Tabla de catálogo que define los niveles de dificultad disponibles en el sistema.
- **Amistad:** Gestión de relaciones entre escaladores, permitiendo crear conexiones entre usuarios.
- **Suscripción:** Control de suscripciones de escaladores a rocódromos específicos.
- **FotosPerfil:** Almacenamiento de imágenes de perfil de escaladores.

5.3.2. Relaciones Principales del Modelo

El modelo relacional de Climbit está estructurado mediante las siguientes relaciones:

Relaciones Jerárquicas: La estructura principal del sistema sigue una jerarquía clara:

- Un Rocódromo contiene múltiples Zonas (relación 1:N).
- Una Zona contiene múltiples Pistas (relación 1:N).

Esta jerarquía refleja la estructura física de los rocódromos reales en la vida.

Relaciones Muchos-a-Muchos: El sistema implementa varias relaciones M:N que requieren tablas de unión:

- Escalador-Escalador a través de Amistad, permitiendo que dos usuarios establezcan relaciones bidireccionales.
- Escalador-Rocódromo a través de Suscripción, permitiendo que un escalador se suscriba a múltiples rocódromos y que un rocódromo tenga múltiples suscriptores.
- Escalador-Pista a través de Ascentso, registrando cada ascenso realizado por cada escalador.

Relaciones de Integridad Referencial: Todas las claves foráneas están configuradas con restricciones de integridad referencial adecuadas para mantener la consistencia de datos, evitando operaciones que dejasen la base de datos en un estado inconsistente.

5.3.3. Normalización y Diseño

La estructura de la base de datos ha sido diseñada siguiendo los principios de normalización relacional, alcanzando la tercera forma normal (3FN) en la mayoría de las entidades. Esto implica:

- Eliminación de datos redundantes, evitando la duplicación de información.
- Cada atributo depende únicamente de la clave primaria de su entidad.
- Separación de conceptos en entidades distintas con relaciones apropiadas entre ellas.

Estas prácticas de normalización garantizan que las operaciones de actualización, inserción y eliminación sean eficientes y no generen inconsistencias en los datos.

5.3.4. Decisiones de Diseño Específicas

A lo largo del diseño de la base de datos tomamos varias decisiones específicas:

Campos de Estado y Booleanos: Utilizamos campos booleanos para representar estados como *Activo* en rocódromos, zonas y pistas, permitiendo desactivar entidades sin eliminarlas completamente de la base de datos, facilitando así el mantenimiento histórico de datos.

Gestión de URLs y Rutas de Imágenes: Campos como *logoURL*, *imagenURL* y *mapa* almacenan rutas a los recursos multimedia asociados a cada entidad. Las imágenes se almacenan en el servidor tras ser procesadas a formato webp mediante ffmpeg, mejorando rendimiento sin perder calidad.

Timestamps de Auditoría: Cada entidad cuenta con campos `FechaCreacion` y, en casos relevantes, `FechaRetirada` o `FechaModificacion` para mantener un registro histórico de cambios en los datos.

Enumeraciones: Ciertos campos utilizan tipos enumerados como `Estado` en ascensos (completado, intentando, etc.) y `Tipo` en pistas (búlder, ruta, etc.), proporcionando validación a nivel de base de datos.

5.3.5. Migraciones y Versionado

La estructura de la base de datos se gestiona a través de migraciones controladas por Sequelize, permitiendo:

- Versionado de cambios en el esquema de la base de datos.
- Posibilidad de revertir cambios si es necesario.
- Documentación implícita de la evolución del modelo de datos.
- Reproducibilidad del esquema en diferentes entornos (desarrollo, pruebas, producción).

Adicionalmente, se utilizan seeds (*seeders*) para poblar la base de datos con datos iniciales como niveles de dificultad estándar, tipos de escalada y datos de prueba, facilitando el desarrollo y las pruebas del sistema.

5.4. Diseño de la UI y UX

Se recogen los criterios de interacción visual, flujo de pantallas y experiencia de uso.

Capítulo 6

Implementación y Desarrollo

Este capítulo documenta la construcción práctica de la solución y las decisiones tomadas durante su desarrollo.

6.1. Desarrollo del Backend

Se describen los módulos principales del servidor, sus responsabilidades y la lógica de negocio implementada.

6.2. Desarrollo del Frontend

Se explica la implementación de la interfaz y la integración con los servicios de la aplicación.

6.3. Desafíos técnicos encontrados y soluciones

Se recogen los problemas técnicos relevantes que aparecieron durante el desarrollo y cómo se resolvieron.

Integración Continua, Despliegue y DevOps

En este capítulo explicamos cómo se ha automatizado la entrega de software y el despliegue de la infraestructura para los diferentes entornos de *ClimbIt*. Se han empleado metodologías de Integración Continua (CI) y Despliegue Continuo (CD) para revisar cada cambio nuevo en el código y de esta manera se pueda publicar de manera segura.

7.1. Diseño la Integración Continua

La integración continua es una práctica que permite fusionar nuevo código desarrollado dentro de producción de forma segura y eficiente. En *ClimbIt* se han implementado dos mecanismos para permitir la integración continua del código.

7.1.1. Calidad de Código y Validación Pre-commits

Antes de subir el código al repositorio remoto, se realiza una primera revisión del código en local para asegurar que cumple con las normas establecidas del proyecto, para ello usamos las siguientes herramientas:

- **Formateo automático con Prettier:** Usamos **Prettier** para que todo el código desarrollado tenga el mismo estilo visual. Se encarga automáticamente de poner puntos y coma al final de las líneas, usar comillas dobles o evitar que las líneas de texto sean demasiado largas.
- **Revisión de errores con ESLint:** **ESLint** es una herramienta que notifica si se están cometiendo errores de lógica o usando malas prácticas al programar en *Javascript*, *Typescript* o *React*. Se ha configurado desactivando alertas innecesarias y avisando si se crean variables que luego no se referencian.
- **Automatización de los pre-commits con Husky:** Usamos una herramienta llamada **Husky** para configurar unos disparadores (*hooks*) de Git que

obligan al código a pasar por Prettier y ESLint justo antes de que el programador realice un commit de los cambios. Si el código falla en la validación de Prettier o EsLint, Husky bloquea el guardado, asegurando de que solo código de buena calidad se sube al repositorio.

7.1.2. Workflow de Integración (Climbit-CI)

Una vez los cambios han pasado las validaciones locales y se suben a GitHub, al crearse una *Pull Request* a una de las ramas principales (`main` o `develop`) mediante GitHub actions se ejecuta la pipeline (*Climbit-CI*) que realiza las siguientes pasos:

- **Creación del entorno:** Se crea en un servidor de GitHub un entorno temporal con la base de datos *PostgreSQL* exclusivamente para hacer las pruebas.
- **Instalación de dependencias:** Dentro del sistema se ejecuta el comando `npm ci` para descargar exactamente las mismas librerías que necesita el proyecto.
- **Pruebas de Integración:** Prepara la base de datos y ejecuta las pruebas automáticas con la herramienta *Jest* para comprobar que toda la lógica de la aplicación y las conexiones (APIs) funcionan correctamente.

7.2. Entornos de desarrollo, preproducción y producción

Para poder desarrollar de forma ágil y segura, se ha dividido el proyecto en tres entornos distintos:

- **Entorno de Desarrollo Local:** Se trata del entorno de desarrollo local en el que los cambios se visualizan al instante (*Hot Reload*). Solo se ejecuta en contenedores de *Docker* la base de datos *PostgreSQL* y su panel de control (*pgAdmin*). El *frontend* y *backend* se ejecuta localmente con el comando `npm run dev`.
- **Entorno de Preproducción (`dev.climbit.es`):** Se ejecuta en un *self-hosted runner* a través GitHub que se define más adelante. Este entorno tiene la finalidad de validar el funcionamiento de los cambios en un servidor real antes de publicarlos en producción (se ejecuta bajo el proyecto `climbit_dev`).
- **Entorno de Producción (`app.climbit.es`):** También se ejecuta en un *self-hosted runner* a través GitHub y se trata la versión final, estable y pública de la aplicación, que es la que usan los usuarios finales. Se ejecuta bajo `climbit_prod`. Es accesible desde cualquier navegador a través de ¹.

¹<https://app.climbit.es/>

7.3. Despliegue de la infraestructura

Para publicar *ClimbIt*, tomamos la decisión no usar servidores *Cloud* ya que realizándolo localmente tenemos un control total sobre la información y además nos ayuda a ahorrar costes. Para ello se ha montado un servidor (*self-hosted*) que se puede utilizar mediante GitHub actions.

7.3.1. Orquestación mediante Contenedores

Tanto el entorno de preproducción como el de desarrollo se organizan usando *Docker Compose*. Esta arquitectura agrupa cuatro contenedores que se conectan entre sí:

1. **Frontend:** Servidor web que entrega la interfaz de usuario.
2. **Backend:** API Node.js que gestiona la lógica de negocio.
3. **PostgreSQL:** Motor de base de datos relacional.
4. **pgAdmin:** Interfaz gráfica para la administración de la base de datos.

7.3.2. Hardware y Ejecución (Self-hosted Runner)

Físicamente, nuestro servidor funciona en una **Raspberry Pi 4 (con 4GB de RAM)** usando el sistema operativo *PiOS* basado en *Debian*. Dentro del servidor se ha instalado un *GitHub Self-hosted Runner* como servicio. Se trata de un programa que espera continuamente órdenes de despliegue automático desde GitHub. Cuando recibe la orden, la Raspberry Pi vuelve a montar la aplicación y le asigna los puertos correctos (el puerto 80 para producción y el 81 para preproducción) de forma automática y segura.

7.3.3. Exposición Segura y Cifrado: Cloudflare Zero Trust

Para acceder al servidor por internet sin poner en peligro nuestra red wifi del domicilio, decidimos usar un túnel seguro llamado **Cloudflare Zero Trust**. Este sistema crea una conexión directa y protegida (*tunnel*) desde las direcciones de internet (`app.climbit.es` y `dev.climbit.es`) en los servidores de CloudFlare hasta nuestra Raspberry Pi.

Toda la información viaja encriptada usando el protocolo **HTTPS** y Cloudflare se encarga automáticamente de gestionar los certificados de seguridad (**certificados SSL/TLS**) en sus propios servidores. De esta forma, se garantiza la seguridad de los datos de los usuarios sin tener que lidiar con configuraciones de seguridad ni tener que abrir puertos.

7.4. Empaquetado y Subida a Google Play

Aunque *ClimbIt* este construida como una página web progresiva (PWA), decidimos subirla también a la Play Store de Android, debido a que el proceso de instalación resultaba lioso para perfiles de bajo nivel tecnológico y subiendola a la play store se reduce ese rozamiento.

7.4.1. Tecnología Trusted Web Activity (TWA) y Bubblewrap

Para transformar una aplicación web en un formato que acepte Android (.aab), usamos un programa llamado **Bubblewrap** desarrollado por Google. Esta herramienta envuelve nuestra web usando una tecnología llamada *Trusted Web Activity* (TWA) y gracias a esto, la aplicación se asimila a cualquier otra aplicación nativa instalada en el móvil, aunque por dentro sigue funcionando conectada a nuestro servidor. Tiene la ventaja de que cualquier cambio que hagamos en la web se actualiza al instante en la aplicación del móvil.

7.4.2. Configuración de Verificación y Requisitos Legales

Para poder publicar en Google Play completamos los siguientes pasos técnicos y legales:

- **Verificación del dominio (*assetlinks.json*):** Se configuro en la ruta (`.well-known/assetlinks`) el envío de información para demostrar a Google que la web es nuestra. Esto permite que el móvil confíe en la aplicación, y no aparezca la barra de búsqueda del navegador y muestre la *app* en pantalla completa.
- **Política de Privacidad:** Redactamos y publicamos una página explicando cómo tratamos los datos ya que es un requisito para subir aplicaciones en Google Play. Para ello se utilizo una plantilla de la página iubenda como se ve en ².
- **Manifiesto de Aplicación Web (*manifest.json*):** Se configuró este archivo con los iconos, colores y datos de la aplicación para que, al instalarla, se integre visualmente en cualquier dispositivo Android.
- **Configuración de el perfil de la aplicación:** Además de esto para la tienda se tuvo que crear una descripción, banner, formulario de eliminación de datos y capturas en dispositivos para la tienda.

7.4.3. Fase de Lanzamiento: Prueba Cerrada

Actualmente *ClimbIt* está en la fase de **Prueba Cerrada** dentro de Google Play Console, por lo que solo un grupo de *testers* tienen acceso para probarla.

²<https://www.iubenda.com/privacy-policy/86624965>

Esto permite ver cómo funciona en diferentes móviles y asegurarnos de que toda la tecnología PWA y TWA responde correctamente antes de lanzarla para todos los usuarios.

Capítulo 8

Pruebas y Validación

Este capítulo reúne la estrategia de verificación de la solución y la validación de la utilidad de la aplicación.

8.1. Estrategia de pruebas técnicas

Se definen los tipos de pruebas aplicadas para comprobar el correcto funcionamiento del sistema.

8.2. Pruebas con usuarios reales (User Testing)

Se describe la validación realizada con usuarios para comprobar usabilidad y adecuación al contexto real.

8.3. Análisis de Resultados y Mejoras aplicadas

Se sintetizan los resultados obtenidos y los cambios introducidos a partir de ellos.

Capítulo 9

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 9.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

Kaya climbing app. 2026.

Moonboard. 2026.

Pokémon go. 2026.

Strava. 2026.

Toplogger. 2026.

Weclimb. 2026.

Apéndice **A**

Título del Apéndice A

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Apéndice	B
----------	----------

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

